

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Catherine Patricia Sindunata - 5024241040

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Mempersiapkan project modul 1 untuk STM32 yang sudah dibuat sebelumnya dan mengubah isi dari `platformio.ini` menjadi

```
[env:blackbpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Menghubungkan ST-Link ke Blackpill dengan kabel jumper dan menyolokkan ST-Link ke komputer lewat USB. Mencocokkan nama pin di STM32 dan ST-Link agar sesuai dengan urutannya.
3. Menuliskan kode Blink yang sudah disediakan pada `src/main.cpp` dan klik *Build* lalu *Upload*.
4. Memperhatikan LED di board (PC13) akan menyala setengah detik dan mati setengah detik. Proses ini akan berulang terus menerus.

1.2 Percobaan 2: Pengenalan ESP32 dengan *Framework* ESP-IDF (Blink)

1. Mempersiapkan project modul 1 untuk ESP32 yang sudah dibuat sebelumnya dan mengubah isi dari `platformio.ini` menjadi

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Menyambungkan ESP32 ke komputer lewat USB dan memastikan komputer mengenali *port*-nya.
3. Menghapus file `src/main.cpp` dan menambahkan file baru dengan nama `src/main.c`. Menuliskan kode Blink yang telah disediakan lalu klik *Build* dan *Upload*.
4. Memperhatikan LED di board (GPIO2) akan menyala setengah detik dan mati setengah detik. Proses ini akan berulang terus menerus.

1.3 Percobaan 3: *Pull-up* dan *Pull-down*: Eksternal vs Internal (ESP32 + *Framework* Arduino)

1.3.1 Eksperimen 1: *Pull-up* dan *Pull-down* Eksternal

1. Kembali ke project modul 1 untuk ESP32 yang sudah dibuat sebelumnya dan mengubah isi `platformio.ini` menjadi

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

Lalu menghapus file `src/main.c` dan menambahkan file baru dengan nama `src/main.cpp`.

2. Memasang dua tombol di GPIO 32 dan 33 lengkap dengan resistornya sesuai dengan Skema Bagian A.
3. Menuliskan kode yang telah disediakan pada `src/main.cpp` lalu klik *Build* dan *Upload*.
4. Menekan kedua tombol secara bergantian atau bersamaan lalu mencatat hasil yang ditampilkan di *Serial Monitor*.

1.3.2 Eksperimen 2: *Pull-up* dan *Pull-down* Internal

1. Melepas kedua resistor dan menyesuaikan rangkaiannya sesuai Skema Bagian B.
2. Mengganti isi file `src/main.cpp` dengan kode yang telah disediakan lalu klik *Build* dan *Upload*.
3. Menekan kedua tombol secara bergantian atau bersamaan lalu mencatat hasil yang ditampilkan di *Serial Monitor*.

1.4 Percobaan 4: *Debouncing* pada Input Tombol GPIO (ESP32 + *Framework* Arduino)

1. Masih menggunakan konfigurasi `platformio.ini` yang sama dengan percobaan sebelumnya, rangkaian untuk percobaan keempat dirangkai sesuai dengan skema yang baru.
2. Menuliskan kode yang telah disediakan pada `src/main.cpp` lalu klik *Build* dan *Upload*.
3. Menekan tombol satu kali, lalu membandingkan nilai penghitung `counterNoDebounce` dan `counterWithDebounce` yang ditampilkan di *Serial Monitor*.
4. Mengubah nilai `DEBOUNCE_DELAY` pada kode menjadi 10 ms, lalu mengulang langkah sebelumnya dan mencatat hasilnya.
5. Mengubah nilai `DEBOUNCE_DELAY` pada kode menjadi 200 ms, lalu mengulang dua langkah sebelumnya dan mencatat hasilnya.

1.5 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + *Framework* Arduino)

1. Masih menggunakan konfigurasi `platformio.ini` yang sama dengan percobaan sebelumnya, rangkaian untuk percobaan kelima dirangkai sesuai dengan skema yang baru. Tidak lupa untuk menghubungkan titik tengah keduanya ke GPIO 34.
2. Menuliskan kode yang telah disediakan pada `src/main.cpp` lalu klik *Build* dan *Upload*.
3. Membaca nilai ADC mentah yang ditampilkan di *Serial Monitor* sambil menutup LDR dengan tangan atau menyinari LDR dengan lampu.

2 Analisis Hasil Percobaan

Praktikum Modul 1 diawali dengan pengenalan dua *board* yang digunakan, yaitu STM32 Blackpill dan ESP32, melalui program Blink. Pada Percobaan 1, STM32 Blackpill diprogram menggunakan *framework* Arduino (STM32duino) dan di-*flash* melalui ST-Link dengan protokol SWD. Program cukup ditulis dalam dua fungsi, yaitu `setup()` untuk mengatur pin PC13 sebagai output menggunakan `pinMode()`, dan `loop()` yang dijalankan berulang secara otomatis untuk menyalakan dan mematikan LED menggunakan `digitalWrite()` dan `delay()`. LED *onboard* berhasil berkedip dengan interval 500 ms. Perlu diperhatikan bahwa LED pada PC13 bersifat aktif LOW, sehingga LED menyala ketika pin diberi logika LOW.

Program yang sama kemudian diimplementasikan pada ESP32 menggunakan *framework* ESP-IDF dalam Percobaan 2, dan dari sini terlihat perbedaan mendasar kedua *framework*. Pada ESP-IDF, program dimulai dari fungsi `app_main()` dan perulangannya harus ditulis sendiri menggunakan `while(1)`. Pin perlu di-*reset* dan diatur arahnya secara eksplisit menggunakan `gpio_reset_pin()` dan `gpio_set_direction()`, level pin diatur menggunakan `gpio_set_level()`, serta *header* FreeRTOS dan *driver* GPIO harus di-*include* secara manual. Fungsi tunda yang digunakan adalah `vTaskDelay(pdMS_TO_TICKS(500))`, yang menyerahkan CPU kepada *task* lain selama menunggu, sedangkan `delay()` pada Arduino memblokir alur program. Program ESP-IDF ditulis dalam berkas `main.c` (bahasa C), dan proses *build* pertamanya jauh lebih lama, yaitu sekitar 390 detik, karena seluruh komponen *framework* ikut dikompilasi. Logika LED kedua *board* juga berkebalikan, karena LED GPIO 2 pada ESP32 bersifat aktif HIGH. Dari kedua percobaan ini dapat disimpulkan bahwa *framework* Arduino lebih ringkas dan mudah dipahami, sedangkan ESP-IDF lebih panjang tetapi memberikan kontrol langsung terhadap perangkat keras dan RTOS.

Percobaan 1 dan 2 menggunakan GPIO sebagai digital *output*. Pada Percobaan 3, GPIO dimanfaatkan sebagai digital *input* untuk membaca status tombol, dengan dua konfigurasi resistor, yaitu eksternal dan internal . Hasil kedua konfigurasi menunjukkan logika yang sama. Pada GPIO 32 dengan konfigurasi *pull-up*, pin terbaca HIGH (TIDAK DITEKAN) ketika tombol tidak ditekan dan LOW (DITEKAN) ketika tombol ditekan. Pada GPIO 33 dengan konfigurasi *pull-down* berlaku kebalikannya, yaitu LOW ketika tidak ditekan dan HIGH ketika ditekan. Pada konfigurasi eksternal, status kedua tombol berganti secara stabil sesuai tombol yang ditekan. Pada konfigurasi internal, status kedua pin sebagian besar TIDAK DITEKAN, dan status DITEKAN hanya muncul sesaat ketika tombol ditekan. Hal ini menunjukkan bahwa resistor internal ESP32 yang diaktifkan melalui `INPUT_PULLUP` dan `INPUT_PULLDOWN` dapat menggantikan resistor eksternal 10 kΩ tanpa mengubah logika program. Perbedaannya, resistor internal ESP32 bernilai lebih besar (sekitar 45 kΩ), sehingga tarikannya lebih lemah dan sedikit lebih rentan terhadap gangguan dibandingkan resistor eksternal.

Saat pengujian kode konfigurasi internal, keluaran *Serial Monitor* awalnya berupa karakter acak (*gibberish*) ketika program menggunakan `Serial.begin(115200)`. Hal ini bukan disebabkan oleh *noise*, melainkan oleh ketidaksesuaian *baud rate* antara ESP32 dan *Serial Monitor*. Berkas `platformio.ini` saat itu belum memuat parameter `monitor_speed`, sehingga *Serial Monitor* PlatformIO membaca data dengan *baud rate* bawaan sebesar 9600 bps, sedangkan ESP32 mengirim data pada 115200 bps. Komunikasi UART bersifat asinkron dan tidak memiliki sinyal *clock* bersama, sehingga pengirim dan

penerima harus menggunakan *baud rate* yang sama agar setiap bit dicuplik pada waktu yang tepat. Jika keduanya berbeda, bit terbaca pada posisi yang salah dan byte yang terbentuk tidak sesuai dengan data aslinya. Untuk mengatasinya, *baud rate* pada program diubah menjadi `Serial.begin(9600)` agar sesuai dengan *Serial Monitor*, dan keluaran pun terbaca dengan benar. Perubahan ini tidak memengaruhi hasil pembacaan tombol, karena hanya menyangkut kecepatan pengiriman data ke komputer. Pada percobaan berikutnya, parameter `monitor_speed = 115200` ditambahkan pada `platformio.ini`, sehingga program lain dapat tetap menggunakan `Serial.begin(115200)` sesuai modul. Dari kejadian ini dapat disimpulkan bahwa *baud rate* pada `Serial.begin()` harus selalu sama dengan `monitor_speed` pada `platformio.ini`.

Resistor *pull-up* dan *pull-down* memastikan pin tidak berada dalam kondisi *floating*, tetapi belum menjamin satu kali penekanan tombol terbaca sebagai satu kejadian. Masalah ini diuji pada Percobaan 4 dengan membandingkan penghitung tanpa *debounce* dan penghitung dengan *debounce* pada tiga nilai `DEBOUNCE_DELAY`. Pada nilai bawaan 50 ms, hasil akhirnya adalah 11 hitungan tanpa *debounce* dan 9 hitungan dengan *debounce*. Artinya, dua hitungan palsu akibat *bouncing* berhasil dibuang, dan tombol tetap responsif terhadap setiap penekanan normal. Ketika nilai diturunkan menjadi 10 ms, penghitung dengan *debounce* hampir selalu mengikuti penghitung tanpa *debounce*, dan keduanya berakhir pada nilai 16. Hal ini menunjukkan bahwa filter 10 ms hampir tidak menyaring apa pun, sehingga pantulan yang berlangsung lebih lama dari 10 ms tetap terhitung sebagai penekanan. Sebaliknya, pada nilai 200 ms, penghitung tanpa *debounce* mencapai 14, sedangkan penghitung dengan *debounce* tetap bernilai 0. Tombol ditekan secara cepat dan berulang, sehingga sinyal LOW tidak pernah bertahan stabil selama 200 ms dan tidak ada satu pun penekanan yang dianggap sah. Hasil ini membuktikan adanya *trade-off*: semakin besar nilai `DEBOUNCE_DELAY`, semakin bersih pembacaan dari *bouncing*, tetapi tombol semakin lambat merespons dan penekanan cepat dapat terlewat sepenuhnya. Pada percobaan ini, nilai sekitar 50 ms merupakan kompromi terbaik.

Percobaan 3 dan 4 masih membaca GPIO sebagai digital input yang hanya mengenal dua kondisi, yaitu HIGH dan LOW. Percobaan 5 memperkenalkan mode akses GPIO ketiga, yaitu analog *input* melalui ADC, menggunakan LDR. LDR dirangkai sebagai pembagi tegangan bersama resistor 10 kΩ, sehingga tegangan pada GPIO34 memenuhi persamaan:

$$V_o = 3{,}3 \times \frac{10\,k\Omega}{R_{LDR} + 10\,k\Omega}$$

Hambatan LDR berubah secara kontinu sesuai intensitas cahaya, sehingga tegangan keluarannya juga berubah secara kontinu dan tidak hanya bernilai 0 V atau 3,3 V. Ketika LDR ditutup, nilai ADC berada pada kisaran 357–583 (0,29–0,47 V), yang berarti hambatan LDR sekitar 100 kΩ. Ketika LDR disinari, nilainya naik menjadi 1867–3049 (1,50–2,46 V), yang berarti hambatan LDR turun hingga sekitar 3,4 kΩ. Di antara kedua kondisi tersebut juga muncul nilai-nilai peralihan. Digital *input* hanya membandingkan tegangan dengan suatu ambang batas sehingga hasilnya terbatas pada HIGH atau LOW. Sementara itu, ADC 12-bit pada ESP32 memetakan rentang 0–3,3 V ke dalam 4096 level (0–4095) dengan selisih tiap level sekitar 0,8 mV, sehingga mampu membedakan ribuan kondisi intensitas cahaya, bukan sekadar gelap atau terang.

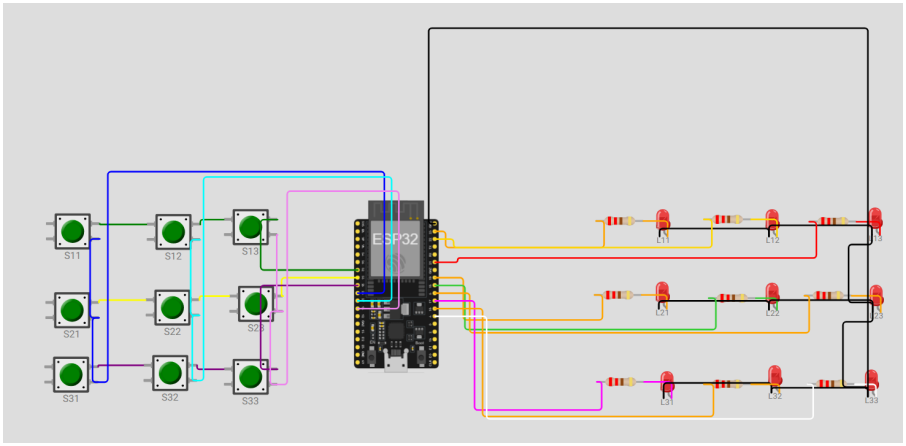
Secara keseluruhan, rangkaian percobaan pada Modul 1 menunjukkan tiga mode akses GPIO secara

bertahap yaitu digital *output* pada program Blink, digital *input* pada pembacaan tombol beserta konfigurasi resistor dan *debouncing*, serta analog *input* pada pembacaan LDR melalui ADC. Percobaan ini juga menunjukkan bahwa keberhasilan sistem tertanam tidak hanya bergantung pada logika program, tetapi juga pada konfigurasi pendukung seperti *framework* yang digunakan, kesesuaian *baud rate* komunikasi serial, dan pemilihan parameter *debounce* yang tepat.

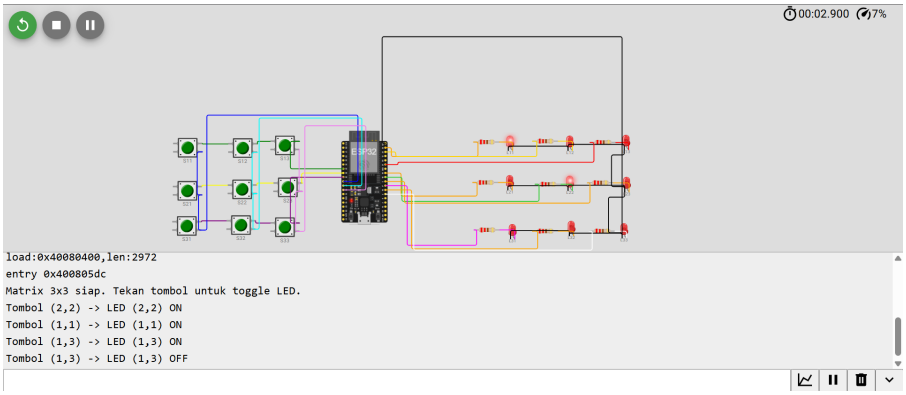
3 Hasil Tugas Modul

3.1 Hasil Simulasi dan Source Code Matrix Pushbutton 3x3 yang Mengendalikan Matriks LED 3x3

3.1.1 Hasil Simulasi



Gambar 1: Rangkaian Simulasi



Gambar 2: Hasil Simulasi

3.1.2 Source Code

```
#include <Arduino.h>

const uint8_t ROW_PINS[3] = {32, 33, 25};
const uint8_t COL_PINS[3] = {26, 27, 14};
const uint8_t LED_PINS[3][3] = {
    {23, 22, 21},
    {19, 18, 5},
```

```
        {17, 16, 4}
    };

    #define DEBOUNCE_DELAY 50

    int lastReading[3][3];
    int stableState[3][3];
    unsigned long lastDebounceTime[3][3];
    bool ledState[3][3];

    void setup() {
        Serial.begin(115200);

        for (int r = 0; r < 3; r++) {
            pinMode(ROW_PINS[r], INPUT);
    }

        for (int c = 0; c < 3; c++) {
            pinMode(COL_PINS[c], INPUT_PULLUP);
    }

        for (int r = 0; r < 3; r++) {
            for (int c = 0; c < 3; c++) {
                pinMode(LED_PINS[r][c], OUTPUT);
                digitalWrite(LED_PINS[r][c], LOW);
                lastReading[r][c] = HIGH;
                stableState[r][c] = HIGH;
                lastDebounceTime[r][c] = 0;
                ledState[r][c] = false;
            }
        }
        Serial.println("Matrix 3x3 siap. Tekan tombol untuk toggle LED.");
    }

    void loop() {
        for (int r = 0; r < 3; r++) {
            pinMode(ROW_PINS[r], OUTPUT);
            digitalWrite(ROW_PINS[r], LOW);
            delayMicroseconds(10);

            for (int c = 0; c < 3; c++) {
                int reading = digitalRead(COL_PINS[c]);

                if (reading != lastReading[r][c]) {
                    lastDebounceTime[r][c] = millis();
                }

                if ((millis() - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
                    if (reading != stableState[r][c]) {
                        stableState[r][c] = reading;
                        if (stableState[r][c] == LOW) {
                            ledState[r][c] = !ledState[r][c];
                            digitalWrite(LED_PINS[r][c], ledState[r][c] ? HIGH:LOW);
                            Serial.printf("Tombol (%d,%d) -> LED (%d,%d) %s\n",
```

```
                                        r + 1, c + 1, r + 1, c + 1,
                                        ledState[r][c] ? "ON" : "OFF");
                    }
                }
            }
            lastReading[r][c] = reading;
        }

        pinMode(ROW_PINS[r], INPUT);
    }
}
```

3.2 Pertanyaan Analisis

1. Analisis mengapa *debouncing* tetap diperlukan pada rangkaian *matrix pushbutton* ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab *bouncing* pada C.5 *Debouncing*!
   Jawab: *Debouncing* tetap diperlukan karena *bouncing* tidak ada hubungannya dengan berapa banyak tombol yang ditekan. Seperti yang telah dijelaskan, *bouncing* berasal dari getaran fisik kontak logam di dalam setiap tombol saat bersentuhan atau terlepas. Jadi satu tombol yang ditekan sendirian pun tetap menghasilkan beberapa transisi HIGH-LOW dalam beberapa milidetik. Program melakukan *scan* seluruh matriks berulang kali dalam waktu yang sangat singkat, sehingga setiap pantulan ikut terbaca sebagai penekanan baru. Karena LED bekerja dengan sistem *toggle*, jumlah pantulan yang genap membuat LED kembali ke kondisi semula, sehingga tombol terlihat tidak merespons. Jumlah pantulan yang ganjil membuat hasil akhirnya acak. Dengan *debouncing* per tombol, status LED baru berubah setelah sinyal stabil selama 50 ms. Hasilnya, satu tekanan selalu dihitung tepat satu kali.

2. Analisis konfigurasi *pull-up* atau *pull-down* (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan *floating*?
   Jawab: Rancangan ini memakai *pull-up* internal (INPUT_PULLUP) pada ketiga pin kolom. Saat tombol tidak ditekan, pin kolom terbaca HIGH karena ditarik ke 3,3 V. Saat tombol pada baris yang sedang aktif (LOW) ditekan, kolom terhubung ke baris tersebut dan terbaca LOW. Baris yang tidak aktif dibuat *high-impedance* (INPUT) agar tidak terjadi *short-circuit* antarbaris jika dua tombol di kolom yang sama ditekan bersamaan. Pin kolom tidak boleh dibiarkan *floating* karena pada kondisi itu tegangannya tidak terdefinisi dan mudah terpengaruh *noise*, misalnya dari perpindahan *scan* antarbaris atau jalur kabel yang panjang. Akibatnya pin bisa terbaca LOW secara acak, sehingga muncul penekanan palsu dan LED berubah sendiri tanpa ada tombol yang ditekan.

3. Andaikan salah satu tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital *input* (tombol, hanya HIGH/LOW) dan sebagai analog *input* (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?
   Jawab: Digital *input* hanya membandingkan tegangan pin dengan suatu ambang batas, sehingga hasilnya hanya HIGH atau LOW. Itu sudah cukup untuk tombol yang memang hanya

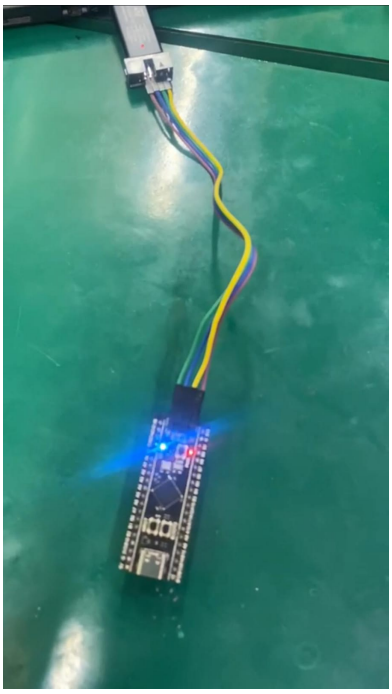
punya dua kondisi, terhubung atau terputus. LDR memiliki perbedaan, yaitu hambatannya berubah secara kontinu sesuai intensitas cahaya, sehingga tegangan pada titik tengah pembagi tegangan juga berubah secara kontinu. ADC 12-bit pada ESP32 mencuplik tegangan tersebut, lalu mengkuantisasinya ke salah satu dari 2^{12} = 4096 level (0–4095), dengan selisih tiap level sekitar 0,8 mV. Karena itu ADC dapat membedakan ribuan tingkat terang, seperti terlihat pada Percobaan 5 (sekitar 357 saat LDR ditutup sampai sekitar 3049 saat disinari). Konsekuensinya, LDR tidak bisa ikut di-*scan* seperti tombol pada matriks. LDR harus dihubungkan ke pin ADC1 tersendiri, misalnya GPIO 34. Kalau ingin LDR berfungsi seperti "tombol", program perlu menentukan nilai ambang sendiri, misalnya LED menyala jika nilai ADC kurang dari 1000.

4 Kesimpulan

Berdasarkan praktikum Modul 1 yang telah dilaksanakan, PlatformIO pada VSCode berhasil digunakan sebagai lingkungan pengembangan untuk dua *board* dan dua *framework* yang berbeda. Program Blink berhasil di-*flash* ke STM32 Blackpill melalui ST-Link dengan *framework* Arduino dan ke ESP32 dengan *framework* ESP-IDF. Dari kedua percobaan tersebut terlihat bahwa Arduino lebih ringkas dan mudah dipahami, sedangkan ESP-IDF lebih panjang tetapi memberi kontrol langsung terhadap perangkat keras dan FreeRTOS. Pada pembacaan tombol, konfigurasi *pull-up* menghasilkan logika HIGH saat tombol tidak ditekan dan LOW saat ditekan, sedangkan *pull-down* menghasilkan kebalikannya. Hasil ini sama, baik dengan resistor eksternal maupun resistor internal ESP32. Praktikum ini juga menunjukkan bahwa *baud rate* pada `Serial.begin()` harus sama dengan `monitor_speed` agar data serial terbaca dengan benar. *Debouncing* terbukti diperlukan untuk menghilangkan hitungan palsu akibat *bouncing*, dengan nilai `DEBOUNCE_DELAY` sekitar 50 ms sebagai kompromi terbaik, di saat nilai 10 ms masih meloloskan pantulan dan nilai 200 ms membuat penekanan cepat tidak terdeteksi sama sekali. Terakhir, pembacaan LDR melalui ADC 12-bit menunjukkan bahwa analog *input* dapat merepresentasikan ribuan level tegangan (0–4095) sesuai intensitas cahaya, berbeda dengan digital *input* yang hanya mengenal dua kondisi. Dengan demikian, praktikum ini telah mencakup tiga mode akses GPIO, yaitu digital *output*, digital *input*, dan analog *input*, yang menjadi dasar untuk modul-modul berikutnya.

5 Lampiran

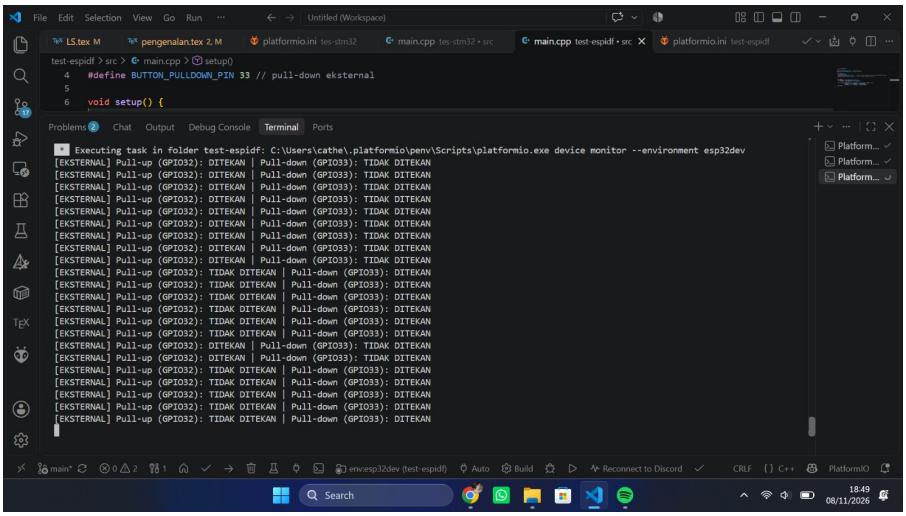
5.1 Dokumentasi saat praktikum



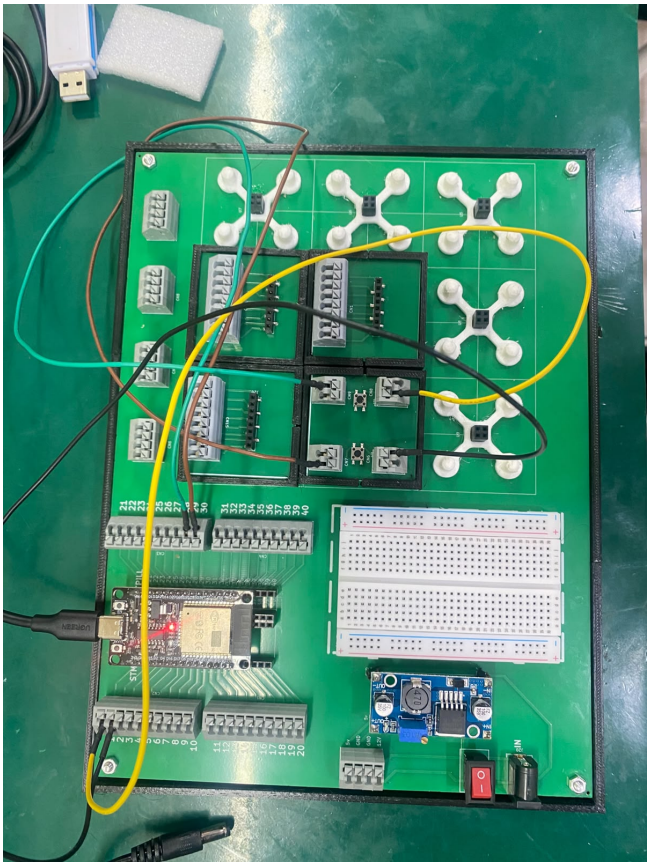
Gambar 3: Hasil Percobaan 1: Blink STM32 Blackpill dengan *framework* Arduino



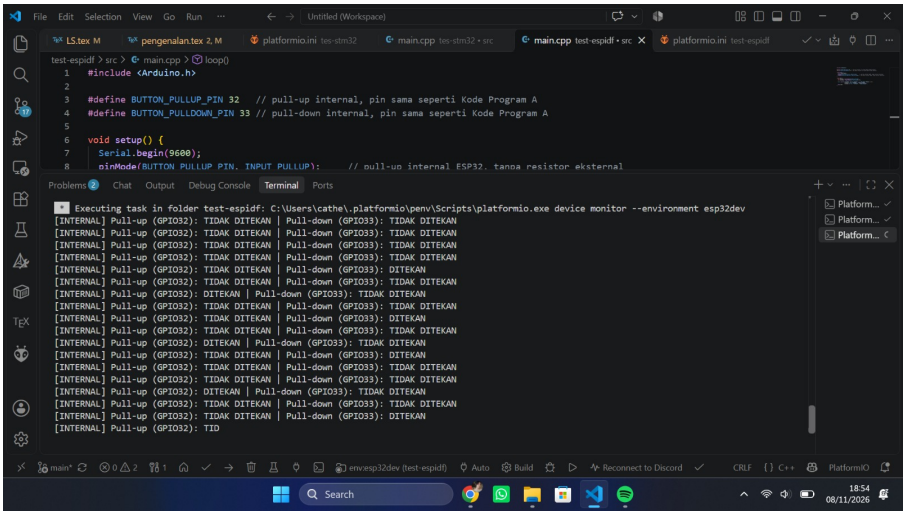
Gambar 4: Hasil Percobaan 2: Blink ESP32 dengan *framework* ESP-IDF



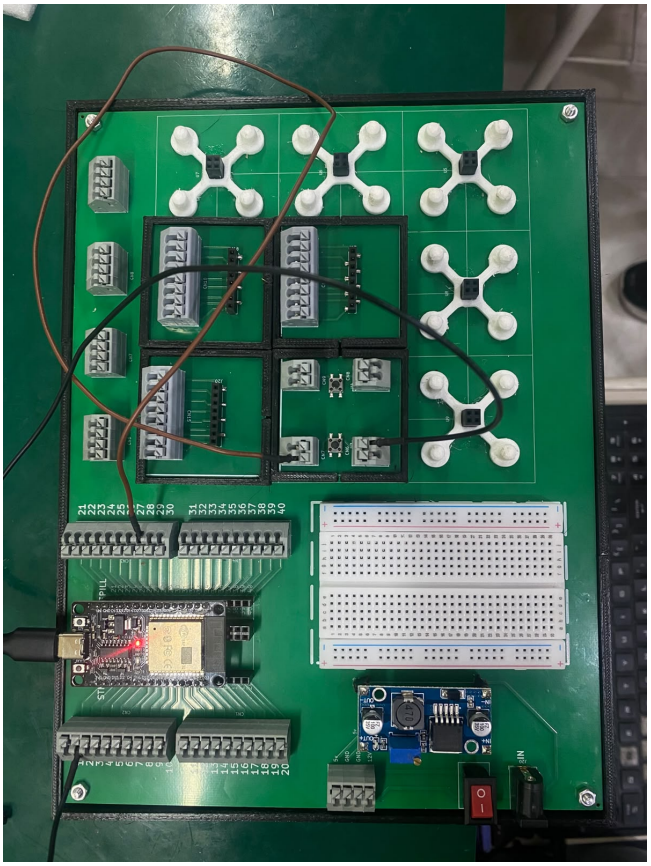
Gambar 5: Hasil Percobaan 3: *Pull-up* dan *Pull-down* Eksternal



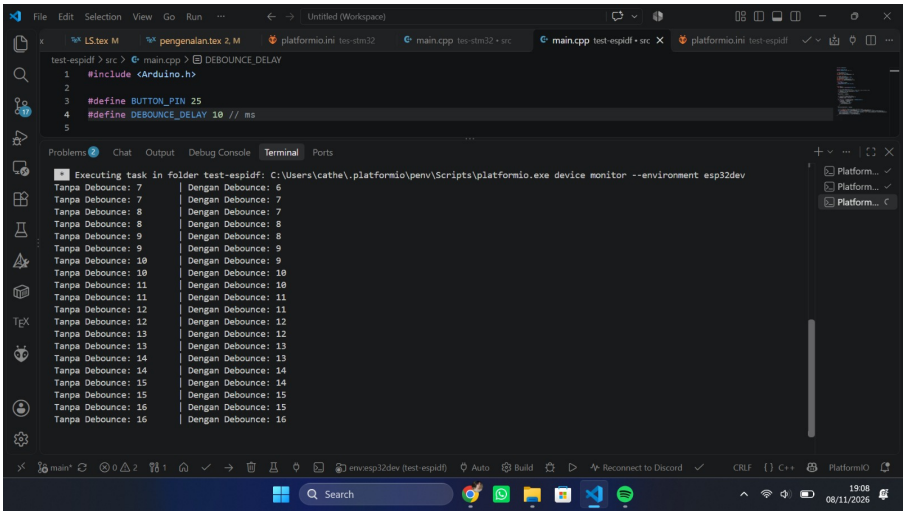
Gambar 6: Rangkaian Percobaan 3: *Pull-up* dan *Pull-down* Internal



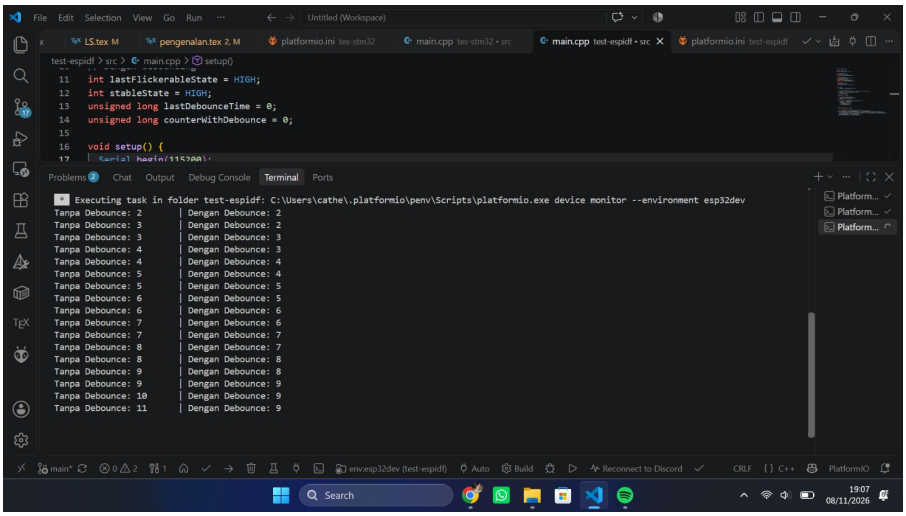
Gambar 7: Hasil Percobaan 3: *Pull-up* dan *Pull-down* Internal



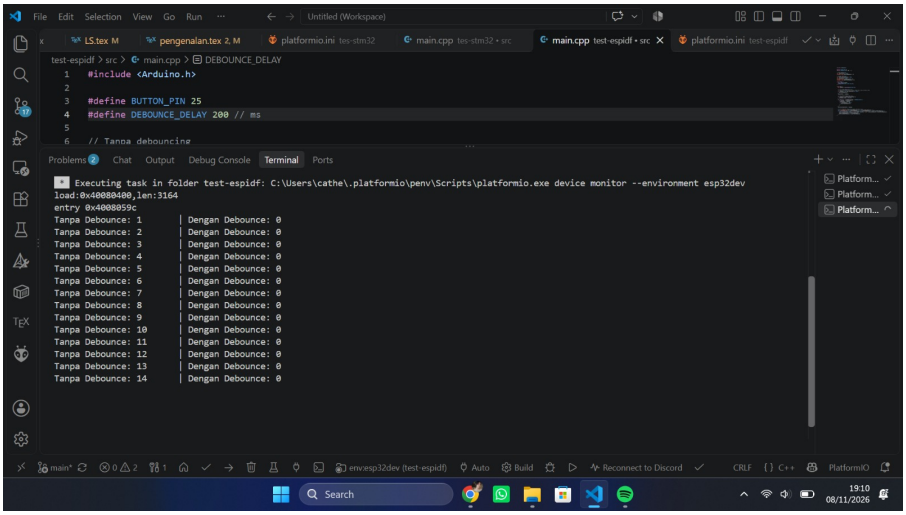
Gambar 8: Rangkaian Percobaan 4: *Debouncing* pada *Input* Tombol GPIO



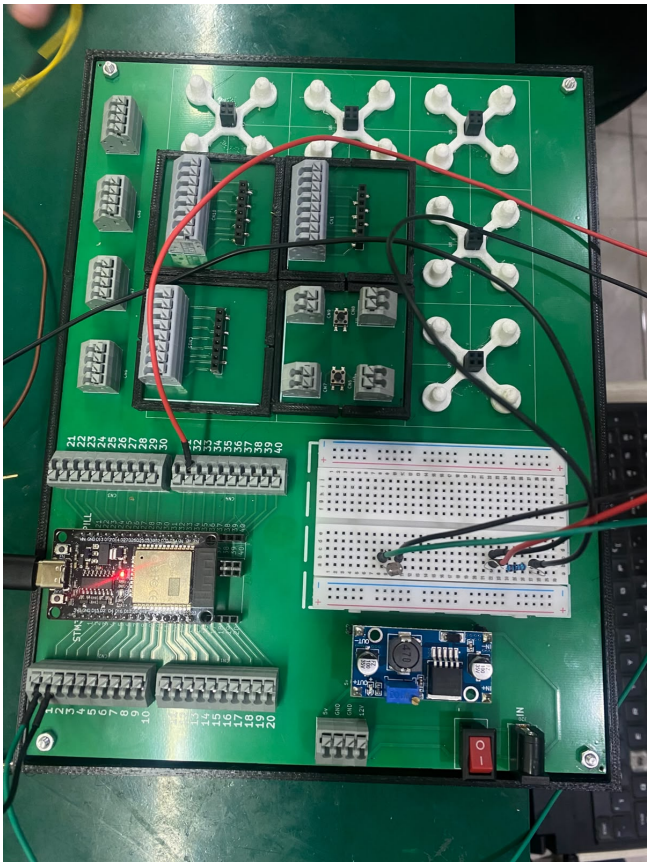
Gambar 9: Hasil Percobaan 4: *Debouncing* pada *Input* Tombol GPIO dengan `DEBOUNCE_DELAY` = 10 ms



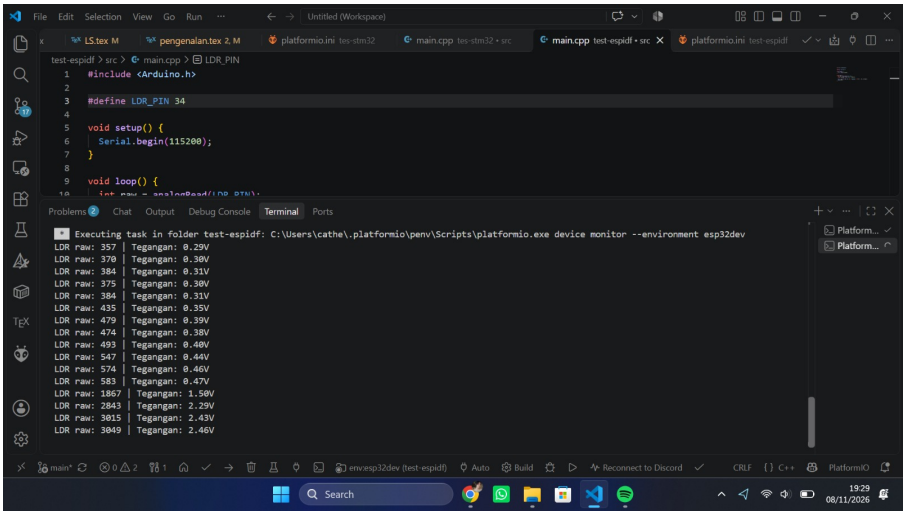
Gambar 10: Hasil Percobaan 4: *Debouncing* pada *Input* Tombol GPIO dengan `DEBOUNCE_DELAY` = 50 ms



Gambar 11: Hasil Percobaan 4: *Debouncing* pada *Input* Tombol GPIO dengan `DEBOUNCE_DELAY` = 200 ms



Gambar 12: Rangkaian Percobaan 5: Pembacaan ADC dengan LDR



Gambar 13: Hasil Percobaan 5: Pembacaan ADC dengan LDR